

Jenkins CI/CD Lab — Practice Answers

ISWE406L — Agile Development Process and DevOps | Windows Agent

Question 1 — Checkout / Install Dependencies / Run Unit Tests

app.py

```
def multiply(a, b):  
    return a * b  
  
def divide(a, b):  
    if b == 0:  
        raise ValueError("Cannot divide by zero")  
    return a / b
```

test_app.py

```
from app import multiply, divide  
  
def test_multiply():  
    assert multiply(3, 4) == 12  
  
def test_divide():  
    assert divide(10, 2) == 5
```

requirements.txt

pytest

Jenkinsfile

```
pipeline {  
    agent { label 'windows' }  
  
    stages {  
        stage('Checkout') {  
            steps {  
                git branch: 'main', url: 'https://github.com/<your-user>/<your-repo>.git'  
            }  
        }  
        stage('Install Dependencies') {  
            steps {  
                bat 'pip install -r requirements.txt'  
            }  
        }  
        stage('Run Unit Tests') {  
            steps {  
                bat 'pytest test_app.py'  
            }  
        }  
    }  
}
```

```

    }
}

post {
    success {
        echo 'BUILD SUCCESSFUL: all unit tests passed.'
    }
    failure {
        echo 'BUILD FAILED: one or more unit tests did not pass.'
    }
}
}

```

Expected result — first run (correct code). All stages run in sequence: the repository is cloned, `pip install -r requirements.txt` succeeds, and `pytest` reports 2 passed. Since every stage completed without error, Jenkins evaluates the `post` block's success condition and prints:

BUILD SUCCESSFUL: all unit tests passed.

After breaking `multiply`. Change the function to return a wrong value, e.g.:

```

def multiply(a, b):
    return a + b  # bug: should be a * b

```

Commit and push this change, then trigger a rebuild. `pytest` now reports 1 passed, 1 failed because `test_multiply` fails its assertion (`multiply(3,4)` now returns 7, not 12). Because the “Run Unit Tests” stage exits with a non-zero status, the whole pipeline is marked **FAILED**, so the `post` block instead runs its failure condition and prints:

BUILD FAILED: one or more unit tests did not pass.

Question 2 — Parametrized Tests (Verbose)

app.py

```

def find_min(numbers):
    return min(numbers)

def count_odds(numbers):
    return sum(1 for n in numbers if n % 2 != 0)

```

test_app.py

```

import pytest
from app import find_min, count_odds

@pytest.mark.parametrize("numbers, expected", [
    ([4, 2, 9, 1], 1),
    ([-3, -7, 5], -7),
    ([10, 10, 10], 10),
])
def test_find_min(numbers, expected):
    assert find_min(numbers) == expected

@pytest.mark.parametrize("numbers, expected", [
    ([1, 2, 3, 4], 2),

```

```

    ([2, 4, 6], 0),
    ([1, 3, 5], 3),
])
def test_count_odds(numbers, expected):
    assert count_odds(numbers) == expected

```

Jenkinsfile

```

pipeline {
    agent { label 'windows' }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/<your-user>/<your-repo>.git'
            }
        }
        stage('Install Dependencies') {
            steps {
                bat 'pip install -r requirements.txt'
            }
        }
        stage('Run Unit Tests (verbose)') {
            steps {
                bat 'pytest -v test_app.py'
            }
        }
    }
}

```

Reading the Console Output. With `-v`, pytest prints one line per parametrized case, not per function, e.g.:

```

test_app.py::test_find_min[numbers0-1] PASSED
test_app.py::test_find_min[numbers1--7] PASSED
test_app.py::test_find_min[numbers2-10] PASSED
test_app.py::test_count_odds[numbers0-2] PASSED
test_app.py::test_count_odds[numbers1-0] PASSED
test_app.py::test_count_odds[numbers2-3] PASSED

```

```

===== 6 passed in 0.05s =====

```

Why 6, not 2. There are only two *test functions*, but `@pytest.mark.parametrize` makes pytest generate one independent *test case* per row of input data. With three rows supplied to each of the two functions, pytest collects and runs $2 \times 3 = 6$ distinct test cases — each shown and counted separately.

Adding a wrong case. Add a case with an incorrect expected value, e.g. `([1, 2, 3], 99)` to `test_find_min`. Rerunning shows 5 passed, 1 failed — the console output marks only that specific parametrized case (`test_find_min[numbers3-99]`) as FAILED, with an assertion error showing the actual vs. expected value, while the other five cases still pass, demonstrating that each parametrized case is evaluated independently.

Question 3 — Environment Variables + Manual Approval Gate

app.py

```
def main():
    print("Application is running")

if __name__ == "__main__":
    main()
```

Jenkinsfile

```
pipeline {
    agent { label 'windows' }

    environment {
        APP_NAME      = 'MyApp'
        APP_VERSION   = '1.0.0'
    }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/<your-user>/<your-repo>.git'
            }
        }
        stage('Build') {
            steps {
                bat 'python -m py_compile app.py'
            }
        }
        stage('Deploy') {
            steps {
                input message: "Approve deployment of ${env.APP_NAME} version ${env.APP_VERSION}"
                ok: 'Release'
                bat 'python app.py'
            }
        }
    }
}
```

Outcome when Release is clicked. The pipeline pauses at the input step, showing the prompt “Approve deployment of MyApp version 1.0.0?” with a **Release** button. Clicking **Release** resumes the pipeline, python app.py executes, prints Application is running, and the build finishes as **SUCCESS**.

Outcome when Abort is clicked. Clicking **Abort** immediately throws a FlowInterruptedException; the Deploy stage — and therefore the whole pipeline — is marked **ABORTED** (shown in grey in the stage view), and python app.py never executes because the step never returns control to the following line.

Question 4 — Build Metadata + Linting

app.py

```
def greet(name):
    return f"Hello, {name}!"

if __name__ == "__main__":
    print(greet("World"))
```

Jenkinsfile

```
pipeline {
    agent { label 'windows' }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/<your-user>/<your-repo>.git'
            }
        }
        stage('Show Build Info') {
            steps {
                echo "BUILD_NUMBER: ${env.BUILD_NUMBER}"
                echo "JOB_NAME: ${env.JOB_NAME}"
                echo "WORKSPACE: ${env.WORKSPACE}"
            }
        }
        stage('Run Linter') {
            steps {
                bat 'flake8 app.py'
            }
        }
    }
}
```

Comparing two runs. JOB_NAME and WORKSPACE stay identical across builds — they depend on the job's configured name and its fixed workspace directory on the agent, not on how many times it has run. BUILD_NUMBER increases by one on every run (e.g. 1 → 2) because Jenkins assigns it as a strictly incrementing counter per job.

After adding an unused import. Add e.g. `import os` to `app.py` without using it, then rebuild. flake8 now reports a style violation such as:

```
app.py:1:1: F401 'os' imported but unused
```

Because flake8 returns a non-zero exit code when it finds violations, the Run Linter stage — and the overall pipeline — fails.

Question 5 — Choice and Boolean Parameters with when

Jenkinsfile

```
pipeline {
    agent { label 'windows' }

    parameters {
        choice(name: 'ENVIRONMENT', choices: ['dev', 'staging', 'prod'], description: 'Target environment')
        booleanParam(name: 'RUN_EXTRA_CHECK', defaultValue: false, description: 'Run the extra checks')
```

```

}

stages {
    stage('Checkout') {
        steps {
            git branch: 'main', url: 'https://github.com/<your-user>/<your-repo>.git'
        }
    }
    stage('Show Parameter') {
        steps {
            echo "Selected environment: ${params.ENVIRONMENT}"
        }
    }
    stage('Extra Check') {
        when {
            expression { return params.RUN_EXTRA_CHECK }
        }
        steps {
            echo 'Running extra verification checks...'
        }
    }
}
}
}

```

Why the first build has no “Build with Parameters” option. Jenkins only knows a pipeline is parameterized after it has parsed the Jenkinsfile. On the very first run there is no prior parsed definition to read, so Jenkins triggers a plain **Build Now**. That first run parses the parameters block and registers it with the job; from the *second* build onward, Jenkins shows **Build with Parameters** instead.

Comparing runs. With RUN_EXTRA_CHECK checked (true), the pipeline view shows all three stages executed, including a green Extra Check box with the “Running extra verification checks...” message. With it unchecked (false), the when condition evaluates to false, so Extra Check is skipped — it appears in the stage view greyed out / marked as skipped rather than run, while Checkout and Show Parameter still execute normally.

Question 6 — Parallel Stages + Artifact Archiving

frontend_check.py

```

import time

time.sleep(4)
with open("frontend_report.txt", "w") as f:
    f.write("Frontend check passed\n")
print("Frontend check complete")

```

backend_check.py

```

import time

time.sleep(4)
with open("backend_report.txt", "w") as f:
    f.write("Backend check passed\n")

```

```
print("Backend check complete")
```

Jenkinsfile

```
pipeline {
  agent { label 'windows' }

  stages {
    stage('Checkout') {
      steps {
        git branch: 'main', url: 'https://github.com/<your-user>/<your-repo>.git'
      }
    }
    stage('Parallel Checks') {
      parallel {
        stage('Frontend Check') {
          steps {
            bat 'python frontend_check.py'
          }
        }
        stage('Backend Check') {
          steps {
            bat 'python backend_check.py'
          }
        }
      }
    }
    stage('Archive Reports') {
      steps {
        archiveArtifacts artifacts: '*_report.txt', fingerprint: true
      }
    }
  }
}
```

Timing estimate. Each script individually sleeps ~4 seconds. Run sequentially, the combined stage would take roughly **8 seconds** (4 + 4) plus a small overhead. Run inside `parallel {}`, both branches start together on separate executors, so the `Parallel Checks` stage takes roughly **4 seconds** — the duration of the *slower* of the two branches, not their sum.

Artifact retention across builds. After a build runs, its `frontend_report.txt` and `backend_report.txt` are stored under that specific build's record (visible from the build's own page under "Build Artifacts"). Triggering a new build creates a **new**, separate artifact record for the new build number; it does not overwrite or delete the previous one. Opening an older build number in the build history still shows its own archived artifacts available for download, confirming Jenkins keeps per-build artifact copies (until a retention/discard policy removes them).

Question 7 — Milestones and Notifications

app.py

```
def main():
    print("Notification demo app running")
```

```
if __name__ == "__main__":
    main()
```

Jenkinsfile

```
pipeline {
    agent { label 'windows' }

    stages {
        stage('Build') {
            steps {
                bat 'python -m py_compile app.py'
                sleep(time: 15, unit: 'SECONDS')
                milestone(1)
            }
        }
        stage('Send Notification') {
            steps {
                script {
                    def subject = "Build #${env.BUILD_NUMBER} - ${env.JOB_NAME}"
                    def body = "Build URL: ${env.BUILD_URL}"
                    try {
                        mail to: 'you@example.com',
                            subject: subject,
                            body: body
                    } catch (err) {
                        echo "SMTP not configured – notification content:"
                        echo "Subject: ${subject}"
                        echo "Body: ${body}"
                    }
                }
            }
        }
    }
}
```

(Note: Checkout is omitted from this snippet only where `app.py` already exists on the agent; add a `git` step back in as stage one if pulling from source control, as in the earlier questions.)

Clicking Build Now twice in quick succession. The `milestone(1)` step enforces ordering: only the *newest* build is allowed to pass a given milestone; any older, still-running build that has not yet reached that milestone is automatically cancelled the moment a newer build reaches it. So if build #1 is started and, before it finishes its 15-second sleep, build #2 is started, build #2 will reach `milestone(1)` first (or at least not later) and Jenkins aborts build #1 — it never proceeds to `Send Notification` and therefore never sends a notification. Only the newer build (#2) continues on to send its notification.

After introducing a syntax error in `app.py`. `python -m py_compile app.py` now exits with a non-zero status, so the `Build` stage fails *before* it reaches `milestone(1)` or `sleep`. Because a pipeline's default behavior is to stop the remaining stages once a stage fails, the `Send Notification` stage is never executed — no `mail` step runs — so no email (or `echo` fallback) is produced at all.